

LAPORAN PRAKTIKUM 4
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT) QUEUE*
MENGGUNAKAN *ARRAY*



Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Laporan Praktikum 4 Struktur Data: Implementasi Abstract Data Type (ADT) Queue Menggunakan Array” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep Abstract Data Type (ADT), khususnya pada struktur data Queue, serta mengimplementasikannya menggunakan Array dan ArrayList dalam bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu memahami cara kerja antrian data dengan prinsip First In First Out (FIFO) dalam penyelesaian suatu permasalahan.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan Queue dalam pemrograman.

Padang, 29 April 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu komputer yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah Queue atau antrian. Queue adalah struktur data yang menerapkan prinsip First In First Out (FIFO), yaitu data yang pertama kali masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian di kasir, antrian kendaraan, maupun sistem antrian pada komputer.

Dalam pemrograman, Queue sangat berguna untuk mengatur proses yang berjalan secara berurutan. Implementasi Queue dapat dilakukan dengan berbagai cara, salah satunya menggunakan Array. Penggunaan Array dalam Queue dipilih karena memiliki susunan data yang teratur dan mudah dipahami, sehingga cocok digunakan untuk mempelajari dasar konsep antrian data. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami cara kerja Queue menggunakan Array serta mampu mengimplementasikannya ke dalam bahasa pemrograman Java.

1.2 Tujuan

1. Memahami konsep dasar struktur data Queue.
2. Mengimplementasikan Queue menggunakan Array dalam bahasa pemrograman Java.
3. Mengetahui prinsip kerja Queue dengan metode FIFO (First In First Out).
4. Melatih kemampuan pemrograman dalam menyelesaikan masalah menggunakan struktur data Queue.

1.3 Manfaat

1. Menambah pemahaman tentang struktur data Queue.
2. Menjadi dasar pembelajaran untuk struktur data lain yang lebih kompleks.
3. Membantu memahami pengelolaan data secara berurutan dalam program.
4. Meningkatkan kemampuan dalam mengimplementasikan Queue menggunakan Array.

BAB 2

PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengorganisasi data agar dapat digunakan secara efisien. Salah satu konsep penting dalam struktur data adalah Abstract Data Type (ADT). ADT adalah suatu tipe data abstrak yang mendefinisikan sekumpulan nilai dan operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan bagaimana implementasinya secara detail.

Konsep utama dari ADT adalah pemisahan antara interface (apa yang dapat dilakukan) dan implementasi (bagaimana cara melakukannya). Dengan menggunakan ADT, programmer dapat lebih fokus pada fungsi dan kegunaan suatu tipe data tanpa harus memahami detail internalnya. Hal ini membuat program menjadi lebih modular, mudah dipelihara, dan mudah dikembangkan.

Dalam bahasa pemrograman Java, ADT biasanya direpresentasikan menggunakan class. Sebuah class berisi atribut (data) dan metode (fungsi) yang mendefinisikan perilaku dari objek tersebut. Atribut digunakan untuk menyimpan data, sedangkan metode digunakan untuk melakukan operasi terhadap data tersebut.

Pada praktikum ini, ADT yang diimplementasikan adalah ADT Queue menggunakan struktur data Array. Queue merupakan struktur data linear yang menerapkan prinsip FIFO (First In First Out), yaitu elemen pertama yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Struktur ini digunakan untuk mengelola data dengan mekanisme penambahan data dilakukan di bagian belakang (rear) dan penghapusan data dilakukan di bagian depan (front).

Selain itu, ADT Queue juga dilengkapi dengan beberapa operasi, antara lain:

1. Penambahan data (enqueue), yaitu menambahkan elemen ke bagian belakang queue.
2. Penghapusan data (dequeue), yaitu menghapus elemen dari bagian depan queue.
3. Melihat elemen terdepan (front/peek), yaitu menampilkan elemen paling depan tanpa menghapusnya.
4. Melihat elemen terakhir (rear), yaitu menampilkan elemen paling belakang pada queue.
5. Menampilkan seluruh isi queue (display), yaitu menampilkan semua data yang tersimpan dalam queue secara berurutan.

6. Mengetahui jumlah elemen (size), yaitu menghitung banyak data yang sedang tersimpan dalam queue.
7. Pengecekan kondisi (isEmpty dan isFull), yaitu untuk mengetahui apakah queue dalam keadaan kosong atau penuh.

Dalam implementasinya, Queue menggunakan Array yang bersifat statis, sehingga jumlah elemen yang dapat disimpan telah ditentukan sejak awal. Penggunaan Array memiliki keunggulan dalam kecepatan akses data dan penggunaan memori yang efisien. Namun, kapasitas Array terbatas sehingga perlu pengelolaan indeks front dan rear dengan baik agar proses penambahan dan penghapusan data dapat berjalan dengan benar.

Dengan memahami dan mengimplementasikan ADT Queue menggunakan Array, mahasiswa dapat mengembangkan kemampuan dalam mengelola data secara terstruktur serta meningkatkan pemahaman terhadap konsep dasar struktur data dalam pemrograman Java.

1. Langkah Kerja

1.1. QueueArray_2511531014

- Kode Program

```
1 package pekan4_2511531014;
2
3 public class QueueArray_2511531014 {
4     int front, rear, size;
5     int capacity;
6     int array[];
7
8     public QueueArray_2511531014(int capacity) {
9         this.capacity = capacity;
10        front = this.size = 0;
11        rear = capacity - 1;
12        array = new int [this.capacity];
13    }
14
15    boolean isFull_2511531014 (QueueArray_2511531014 queue) {
16        return (queue.size == queue.capacity);
17    }
18
19    boolean isEmpty_2511531014 (QueueArray_2511531014 queue) {
20        return (queue.size == 0);
21    }
22
23    void enqueue_2511531014(int item) {
24        if (isFull_2511531014(this))
25            return;
26        this.rear = (this.rear + 1) % this.capacity;
27        this.array[this.rear] = item;
28        this.size = this.size + 1;
29        System.out.println(item + " enqueued to queue");
30    }
31 }
```

```

32● int dequeue_2511531014() {
33     if (isEmpty_2511531014(this))
34         return Integer.MIN_VALUE;
35     int item = this.array[this.front];
36     this.front = (this.front + 1) % this.capacity;
37     this.size = this.size - 1;
38     return item;
39 }
40
41● int front_2511531014() {
42     if (isEmpty_2511531014(this))
43         return Integer.MIN_VALUE;
44
45     return this.array[this.front];
46 }
47
48● int rear_2511531014() {
49     if (isEmpty_2511531014(this))
50         return Integer.MIN_VALUE;
51     return this.array[this.rear];
52 }
53
54 //mencetak elemen antrian
55● void display_2511531014() {
56     int i;
57     if (front == rear) {
58         System.out.printf("\nAntrian Kosong\n");
59         return;
60     }
61
62     //kunjungi dari blkg lalu cetak
63     for (i = front; i < rear; i++) {
64         System.out.printf("%d <--|", array[i]);
65     }
66     return;
67 }
68 }
69

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class QueueArray_2511531014 yang digunakan untuk mengimplementasikan struktur data Queue menggunakan array. Queue merupakan struktur data linear yang menerapkan prinsip FIFO (*First In First Out*), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

1. Class ini memiliki atribut front, rear, dan size yang digunakan untuk mengatur posisi elemen dalam antrian. Variabel front menunjukkan posisi elemen terdepan, rear menunjukkan posisi elemen terakhir, dan size menunjukkan jumlah elemen yang tersimpan. Selain itu, atribut capacity digunakan untuk menentukan kapasitas maksimum queue, sedangkan array[] digunakan sebagai media penyimpanan data.

```

3 public class QueueArray_2511531014 {
4     int front, rear, size;
5     int capacity;
6     int array[];
7

```

2. Constructor QueueArray_2511531014(int capacity) digunakan untuk menginisialisasi queue dengan kapasitas tertentu. Pada constructor ini, nilai front dan size diatur menjadi 0, rear diatur ke capacity - 1, serta array dibuat sesuai kapasitas yang diberikan.

```

8● public QueueArray_2511531014(int capacity) {
9     this.capacity = capacity;
10    front = this.size = 0;
11    rear = capacity - 1;
12    array = new int [this.capacity];
13 }
14

```

3. Method `isFull_2511531014(QueueArray_2511531014 queue)` digunakan untuk mengecek apakah queue dalam kondisi penuh, yaitu ketika jumlah elemen sama dengan kapasitas maksimum. Method `isEmpty_2511531014(QueueArray_2511531014 queue)` digunakan untuk mengecek apakah queue kosong, yaitu ketika jumlah elemen bernilai 0.

```
15 boolean isFull_2511531014 (QueueArray_2511531014 queue) {  
16     return (queue.size == queue.capacity);  
17 }  
18  
19 boolean isEmpty_2511531014 (QueueArray_2511531014 queue) {  
20     return (queue.size == 0);  
21 }  
22
```

4. Method `enqueue_2511531014(int item)` digunakan untuk menambahkan elemen ke dalam queue. Sebelum penambahan, dilakukan pengecekan apakah queue penuh. Jika masih tersedia ruang, posisi rear diperbarui menggunakan operasi modulus agar membentuk *circular queue*, kemudian elemen dimasukkan ke array dan nilai size ditambah satu.

```
2  
3 void enqueue_2511531014(int item) {  
4     if (isFull_2511531014(this))  
5         return;  
6     this.rear = (this.rear + 1) % this.capacity;  
7     this.array[this.rear] = item;  
8     this.size = this.size + 1;  
9     System.out.println(item + " enqueued to queue");  
0 }  
1
```

5. Method `dequeue_2511531014()` digunakan untuk mengambil sekaligus menghapus elemen terdepan dari queue. Jika queue kosong, method mengembalikan `Integer.MIN_VALUE` sebagai penanda. Jika tidak kosong, elemen pada posisi front diambil, kemudian posisi front digeser ke indeks berikutnya menggunakan modulus dan nilai size dikurangi satu.

```
9 int dequeue_2511531014() {  
10     if (isEmpty_2511531014(this))  
11         return Integer.MIN_VALUE;  
12     int item = this.array[this.front];  
13     this.front = (this.front + 1) % this.capacity;  
14     this.size = this.size - 1;  
15     return item;  
16 }
```

6. Method `front_2511531014()` digunakan untuk melihat elemen paling depan tanpa menghapusnya, sedangkan method `rear_2511531014()` digunakan untuk melihat elemen paling belakang. Kedua method ini terlebih dahulu memeriksa apakah queue kosong.

```

    int front_2511531014() {
        if (isEmpty_2511531014(this))
            return Integer.MIN_VALUE;

        return this.array[this.front];
    }

    int rear_2511531014() {
        if (isEmpty_2511531014(this))
            return Integer.MIN_VALUE;
        return this.array[this.rear];
    }
}

```

7. Method `display_2511531014()` digunakan untuk menampilkan seluruh isi antrian. Jika queue kosong, program menampilkan pesan bahwa antrian kosong. Jika tidak kosong, elemen dicetak mulai dari posisi front menuju rear.

```

//mencetak elemen antrian
void display_2511531014() {
    int i;
    if (front == rear) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    //kunjungi dari blkg lalu cetak
    for (i = front; i < rear; i++) {
        System.out.printf(" %d <--", array[i]);
    }
    return;
}
}

```

8. Untuk menjalankan program `QueueArray_2511531014` ini harus dilanjutkan dengan Class Driver `QueueArrayDriver_2511531014`.

1.2. QueueArrayDriver_2511531014

- Kode Program

```

1 package pekan4_2511531014;
2
3 public class QueueArrayDriver_2511531014 {
4     public static void main(String[] args) {
5         QueueArray_2511531014 queue = new QueueArray_2511531014(1000);
6         queue.enqueue_2511531014(10);
7         queue.enqueue_2511531014(20);
8         queue.enqueue_2511531014(30);
9         queue.enqueue_2511531014(40);
10        System.out.println("item di depan " + queue.front_2511531014());
11        System.out.println("item paling belakang " + queue.rear_2511531014());
12        System.out.println("tampilkan queue");
13        queue.display_2511531014();
14        System.out.println();
15        System.out.println(queue.dequeue_2511531014() + "dihapus dari queue");
16        System.out.println("item di depan " + queue.front_2511531014());
17        System.out.println("item paling belakang " + queue.rear_2511531014());
18        System.out.println("tampilkan queue setelah satu data dihapus");
19        queue.display_2511531014();
20    }
21 }
22
23 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class `QueueArrayDriver_2511531014` yang berfungsi sebagai driver untuk menguji implementasi struktur data Queue berbasis array pada class `QueueArray_2511531014`. Pada method `main`, dibuat objek `queue` dengan kapasitas 1000 elemen menggunakan constructor `QueueArray_2511531014(1000)`.

1. Method `enqueue_2511531014()` digunakan untuk menambahkan data ke dalam queue, yaitu nilai 10, 20, 30, dan 40 secara berurutan. Proses ini menunjukkan prinsip FIFO (*First In First Out*), di mana data yang pertama masuk akan berada di posisi depan antrian.

```
public class QueueArrayDriver_2511531014 {
    public static void main(String[] args) {
        QueueArray_2511531014 queue = new QueueArray_2511531014(1000);
        queue.enqueue_2511531014(10);
        queue.enqueue_2511531014(20);
        queue.enqueue_2511531014(30);
        queue.enqueue_2511531014(40);
    }
}
```

2. Method `front_2511531014()` digunakan untuk menampilkan elemen yang berada di posisi depan queue tanpa menghapusnya, sedangkan method `rear_2511531014()` digunakan untuk menampilkan elemen yang berada di posisi paling belakang.

```
System.out.println("item di depan " + queue.front_2511531014());
System.out.println("item paling belakang " + queue.rear_2511531014());
System.out.println("tampilkan queue");
```

3. Method `display_2511531014()` digunakan untuk menampilkan seluruh isi queue dari elemen depan hingga elemen belakang, sehingga pengguna dapat melihat urutan data yang tersimpan di dalam antrian.

```
queue.display_2511531014();
System.out.println();
```

4. Method `dequeue_2511531014()` digunakan untuk mengambil sekaligus menghapus elemen terdepan dari queue. Setelah satu data dihapus, program kembali menampilkan elemen depan, elemen belakang, serta isi queue terbaru untuk menunjukkan perubahan setelah proses penghapusan.

```
System.out.println(queue.dequeue_2511531014() + "dihapus dari queue");
System.out.println("item di depan " + queue.front_2511531014());
System.out.println("item paling belakang " + queue.rear_2511531014());
System.out.println("tampilkan queue setelah satu data dihapus");
queue.display_2511531014();
}
```

5. Hasil Output dari program

```
Console X
<terminated> QueueArrayDriver_2511531014 [Java Application] C:\Users\naysh\p2\pool\plugins\org.e
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
item di depan 10
item paling belakang 40
tampilkan queue
  10 <-- 20 <-- 30 <--
10dihapus dari queue
item di depan 20
item paling belakang 40
tampilkan queue setelah satu data dihapus
  20 <-- 30 <--
```

1.3. QueueLinkedList_2511531014

- Kode Program

```
1 package pekan4_2511531014;
2 import java.util.LinkedList;
3 import java.util.Queue;
4
5 public class QueueLinkedList_2511531014 {
6     public static void main (String[] args) {
7         Queue<Integer> q_1014 = new LinkedList<>();
8         //tambah elemen (0, 1, 2, 3, 4,5) ke antrian
9         for (int i = 0; i < 6; i++)
10             q_1014.add(i);
11         //menampilkan isi antrian
12         System.out.println ("Elemen Antrian " + q_1014);
13         //menghapus kepala antrian
14         int hapus = q_1014.remove();
15         System.out.println ("Hapus Antrian " + hapus);
16         System.out.println (q_1014);
17         //melihat antrian terdepan
18         int depan = q_1014.peek();
19         System.out.println ("Kepala Antrian " + depan);
20
21         int banyak = q_1014.size();
22         System.out.println ("Size Antrian " + banyak);
23     }
24 }
25
26
```

- Pembahasan

Program ini diawali dengan deklarasi package pekan4_2511531014 serta import java.util.LinkedList dan java.util.Queue yang digunakan untuk memanfaatkan struktur data Queue berbasis LinkedList. Program ini dibuat dengan class QueueLinkedList_2511531014 yang berfungsi untuk mendemonstrasikan penggunaan Queue bawaan Java.

1. Pada method main, dibuat objek Queue<Integer> q_1014 = new LinkedList<>() yang digunakan untuk menyimpan data bertipe Integer.

Interface Queue dipadukan dengan implementasi LinkedList sehingga antrian dapat dikelola secara dinamis tanpa batas ukuran tetap seperti array.

```
4
5 public class QueueLinkedList_2511531014 {
6     public static void main (String[] args) {
7         Queue<Integer> q_1014 = new LinkedList<>();
```

2. Struktur perulangan for digunakan untuk menambahkan elemen 0 sampai 5 ke dalam queue menggunakan method add(). Proses ini menempatkan data secara berurutan sesuai prinsip FIFO (*First In First Out*), yaitu elemen pertama yang masuk akan berada di posisi depan antrian.

```
//tambah elemen (0, 1, 2, 3, 4,5) ke antrian
for (int i = 0; i < 6; i++)
    q_1014.add(i);
```

3. Perintah System.out.println("Elemen Antrian " + q_1014) digunakan untuk menampilkan seluruh isi queue setelah proses penambahan data, sehingga urutan elemen dalam antrian dapat dilihat secara langsung.

```
//menampilkan isi antrian
System.out.println ("Elemen Antrian " + q_1014);
```

4. Method remove() digunakan untuk mengambil sekaligus menghapus elemen yang berada di kepala atau bagian depan queue. Nilai yang dihapus disimpan ke variabel hapus, kemudian ditampilkan ke layar bersama isi queue terbaru setelah penghapusan.

```
//menghapus kepala antrian
int hapus = q_1014.remove();
System.out.println ("Hapus Antrian " + hapus);
System.out.println (q_1014);
```

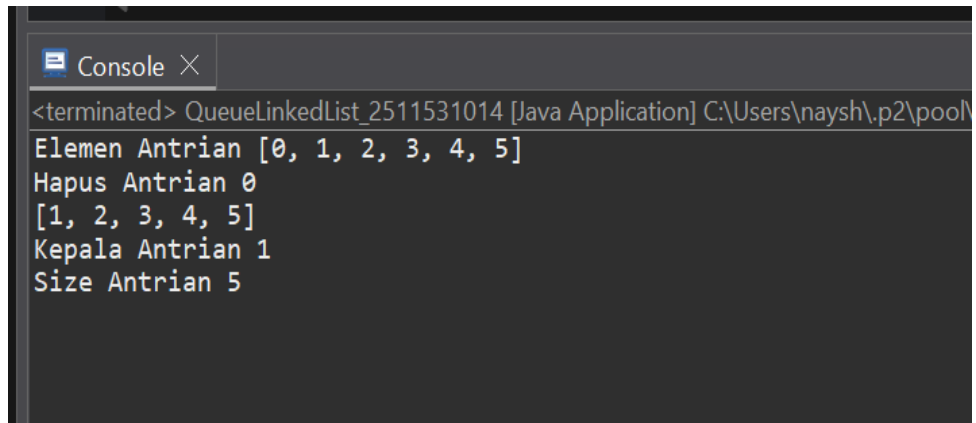
5. Method peek() digunakan untuk melihat elemen terdepan tanpa menghapusnya. Nilai tersebut disimpan ke variabel depan dan ditampilkan sebagai kepala antrian saat ini.

```
System.out.println (q_1014);
//melihat antrian terdepan
int depan = q_1014.peek();
System.out.println ("Kepala Antrian " + depan);
```

6. Method size() digunakan untuk menghitung jumlah elemen yang tersisa di dalam queue setelah proses penghapusan, kemudian hasilnya disimpan ke variabel banyak dan ditampilkan ke layar.

```
int banyak = q_1014.size();
System.out.println ("Size Antrian " + banyak);
}
```

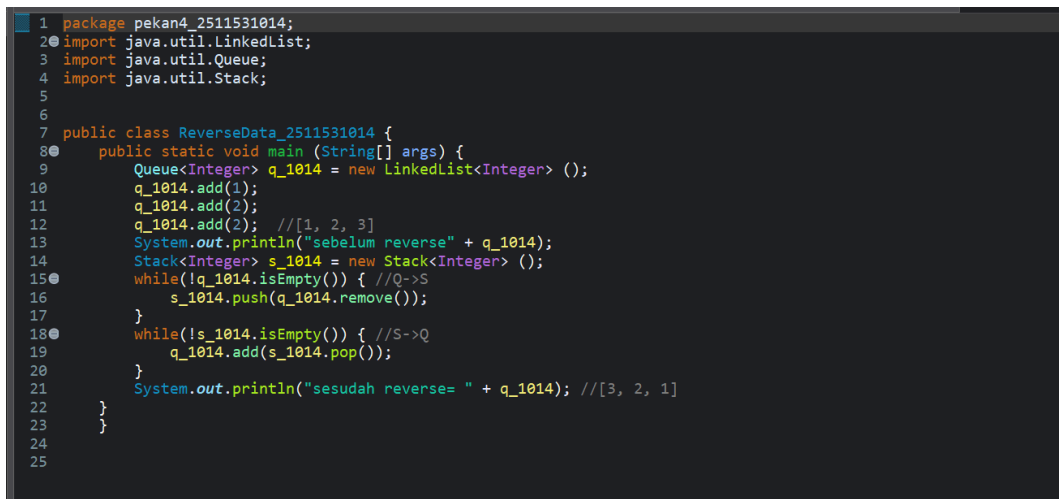
7. Hasil Output Program



```
Console X
<terminated> QueueLinkedList_2511531014 [Java Application] C:\Users\naysh\p2\pool\
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus Antrian 0
[1, 2, 3, 4, 5]
Kepala Antrian 1
Size Antrian 5
```

1.4. ReverseData_2511531014

- Kode Program



```
1 package pekan4_2511531014;
2 import java.util.LinkedList;
3 import java.util.Queue;
4 import java.util.Stack;
5
6
7 public class ReverseData_2511531014 {
8     public static void main (String[] args) {
9         Queue<Integer> q_1014 = new LinkedList<Integer> ();
10        q_1014.add(1);
11        q_1014.add(2);
12        q_1014.add(2); // [1, 2, 3]
13        System.out.println("sebelum reverse" + q_1014);
14        Stack<Integer> s_1014 = new Stack<Integer> ();
15        while(!q_1014.isEmpty()) { //Q->S
16            s_1014.push(q_1014.remove());
17        }
18        while(!s_1014.isEmpty()) { //S->Q
19            q_1014.add(s_1014.pop());
20        }
21        System.out.println("sesudah reverse= " + q_1014); // [3, 2, 1]
22    }
23 }
24
25
```

- Pembahasan

Program ini diawali dengan deklarasi package pekan4_2511531014 serta import java.util.LinkedList, java.util.Queue, dan java.util.Stack yang digunakan untuk memanfaatkan struktur data Queue dan Stack. Program ini dibuat dengan class ReverseData_2511531014 yang berfungsi untuk membalik urutan data pada queue menggunakan bantuan stack.

1. Pada method main, dibuat objek Queue<Integer> q_1014 = new LinkedList<Integer>() yang digunakan untuk menyimpan data bertipe Integer. Data 1, 2, dan 3 ditambahkan ke dalam queue menggunakan method add(), kemudian isi queue ditampilkan sebelum proses pembalikan data.

```

7 public class ReverseData_2511531014 {
8     public static void main (String[] args) {
9         Queue<Integer> q_1014 = new LinkedList<Integer> ();
10        q_1014.add(1);
11        q_1014.add(2);
12        q_1014.add(3); // [1, 2, 3]
13        System.out.println("sebelum reverse" + q_1014);

```

2. Dibuat objek `Stack<Integer> s_1014 = new Stack<Integer>()` yang digunakan sebagai media sementara dalam proses pembalikan urutan elemen. Stack dipilih karena menerapkan prinsip LIFO (*Last In First Out*), sehingga elemen terakhir yang masuk akan keluar terlebih dahulu.

```

Stack<Integer> s_1014 = new Stack<Integer> ();

```

3. Perulangan `while (!q_1014.isEmpty())` digunakan untuk memindahkan seluruh elemen dari queue ke stack. Data diambil dari queue menggunakan method `remove()` lalu dimasukkan ke stack menggunakan `push()`. Proses ini menyebabkan urutan elemen menjadi terbalik di dalam stack.

```

while(!q_1014.isEmpty()) { //Q->S
    s_1014.push(q_1014.remove());
}

```

4. Perulangan kedua `while (!s_1014.isEmpty())` digunakan untuk memindahkan kembali seluruh elemen dari stack ke queue. Data diambil dari stack menggunakan `pop()` lalu dimasukkan kembali ke queue menggunakan `add()`. Karena stack bekerja dengan prinsip LIFO, maka urutan elemen di queue menjadi terbalik dari urutan semula.

```

while(!s_1014.isEmpty()) { //S->Q
    q_1014.add(s_1014.pop());
}

```

5. Setelah proses selesai, isi queue ditampilkan kembali menggunakan `System.out.println()` untuk menunjukkan hasil pembalikan data.

```

}
System.out.println("sesudah reverse= " + q_1014); //[3, 2, 1]
}
}

```

6. Hasil Output Program

```
Console X
<terminated> ReverseData_2511531014 [Java Application] C:\Users\nays
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]
```

1.5. IterasiQueue_2511531014

- Kode Program

```
1 package pekan4_2511531014;
2 import java.util.Iterator;
3 import java.util.LinkedList;
4 import java.util.Queue;
5
6 public class IterasiQueue_2511531014 {
7     public static void main (String args[])
8     {
9         Queue<String> q_1014 = new LinkedList<>();
10        q_1014.add("Praktikum");
11        q_1014.add("Struktur");
12        q_1014.add("Data");
13        q_1014.add("Algoritma");
14        Iterator<String> iterator = q_1014.iterator();
15        while (iterator.hasNext()) {
16            System.out.print(iterator.next() + " ");
17        }
18    }
19 }
20
21 }
22
```

- Pembahasan

Program ini diawali dengan deklarasi package pekan4_2511531014 serta import java.util.Iterator, java.util.LinkedList, dan java.util.Queue yang digunakan untuk memanfaatkan struktur data Queue serta melakukan proses iterasi data. Program ini dibuat dengan class IterasiQueue_2511531014 yang berfungsi untuk menampilkan elemen queue menggunakan Iterator.

1. Pada method main, dibuat objek Queue<String> q_1014 = new LinkedList<>() yang digunakan untuk menyimpan data bertipe String. Queue diimplementasikan menggunakan LinkedList sehingga data dapat dikelola secara dinamis.

```
Queue<String> q_1014 = new LinkedList<>();
```

2. Method `add()` digunakan untuk menambahkan beberapa elemen ke dalam queue, yaitu "Praktikum", "Struktur", "Data", dan "Algoritma". Data dimasukkan secara berurutan sesuai prinsip FIFO (*First In First Out*).

```
q_1014.add("Praktikum");  
q_1014.add("Struktur");  
q_1014.add("Data");  
q_1014.add("Algoritma");  
Iterator<String> iterator = q_1014.iterator();
```

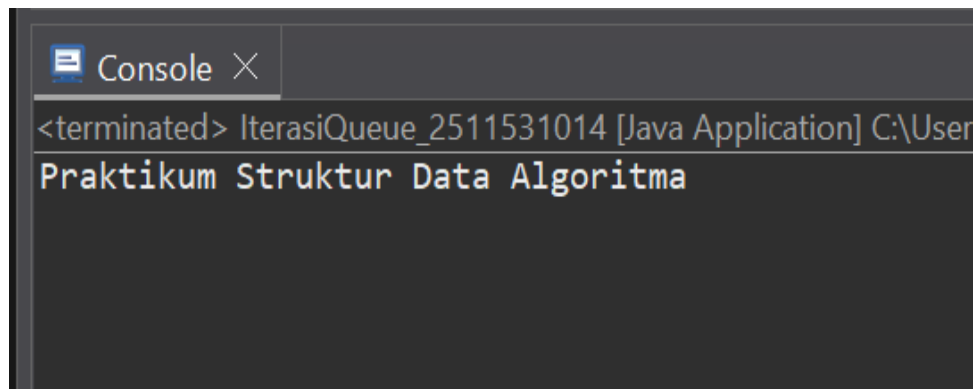
3. Objek `Iterator<String> iterator = q_1014.iterator()` digunakan untuk mengambil iterator dari queue. Iterator berfungsi untuk menelusuri elemen satu per satu tanpa menghapus data yang ada di dalam queue.

```
Iterator<String> iterator = q_1014.iterator();
```

4. Perulangan `while (iterator.hasNext())` digunakan untuk memeriksa apakah masih terdapat elemen berikutnya. Selama masih ada data, method `iterator.next()` digunakan untuk mengambil elemen selanjutnya dan menampilkannya menggunakan `System.out.print()`.

```
while (iterator.hasNext()) {  
    System.out.print(iterator.next() + " ");  
}
```

5. Hasil Output Program



The screenshot shows a console window titled "Console" with a close button. The output text is: `<terminated> IterasiQueue_2511531014 [Java Application] C:\User` followed by the output of the program: `Praktikum Struktur Data Algoritma`.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi struktur data Queue menggunakan array dan LinkedList telah berhasil dilakukan dengan baik serta mampu merepresentasikan konsep Abstract Data Type (ADT) Queue. Kedua metode implementasi tersebut mendukung operasi dasar Queue, seperti penambahan data (*enqueue*), penghapusan data (*dequeue*), melihat elemen terdepan (*front/peek*), serta menampilkan isi antrian, yang semuanya bekerja berdasarkan prinsip FIFO (*First In First Out*).

Pada implementasi menggunakan array, struktur data memiliki ukuran tetap sehingga memerlukan pengelolaan indeks secara manual serta penanganan kondisi penuh dan kosong. Namun, penggunaan array cukup efisien dalam pengaksesan data karena elemen disimpan secara berurutan di memori.

Sedangkan pada implementasi menggunakan LinkedList, struktur data bersifat dinamis sehingga tidak memiliki batasan ukuran tetap dan lebih fleksibel dalam proses penambahan maupun penghapusan elemen. Penggunaan LinkedList juga mempermudah implementasi Queue karena telah tersedia method bawaan Java yang mendukung pengelolaan antrian.

Melalui berbagai program yang telah dibuat, seperti Queue berbasis array, Queue menggunakan LinkedList, pembalikan data Queue menggunakan bantuan Stack, serta iterasi elemen Queue menggunakan Iterator, dapat disimpulkan bahwa struktur data Queue sangat berguna dalam menyelesaikan berbagai permasalahan pemrograman secara sistematis.

Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep Queue, perbedaan implementasi menggunakan array dan LinkedList, serta penerapannya dalam pemrograman Java, sehingga dapat meningkatkan kemampuan dalam merancang dan mengelola struktur data secara efisien, dinamis, dan terstruktur.

TUGAS PEKAN 4

Topik : Implementasi Queue (FIFO)

Tema: antrian pelanggan di loket pelayanan

Nama Class “AntrianLoket_2511531014”

- Kode Program

```
1 package TugasPekan4_2511531014;
2
3 import java.util.Scanner;
4 import java.util.Stack;
5
6 public class AntrianLoket_2511531014 {
7
8     int max = 10;
9     String data[] = new String[max];
10    int front = 0;
11    int rear = -1;
12    int jumlah = 0;
13
14    // Tambah Antrian
15    void enqueue_1014(String isi) {
16        if (isFull_1014()) {
17            System.out.println("Antrian penuh!");
18        } else {
19            rear++;
20            data[rear] = isi;
21            jumlah++;
22            System.out.println("Data berhasil ditambah");
23        }
24    }
25
26    // Hapus Antrian
27    void dequeue_1014() {
28        if (isEmpty_1014()) {
29            System.out.println("Antrian kosong!");
30        } else {
31            System.out.println("Data keluar: " + data[front]);
32
33            for (int i = 0; i < rear; i++) {
34                data[i] = data[i + 1];
35            }
36
37            rear--;
38            jumlah--;
39        }
40    }
41
42    // Tampilkan Antrian
43    void display_1014() {
44        if (isEmpty_1014()) {
45            System.out.println("Antrian kosong!");
46        } else {
47            System.out.println("Isi antrian:");
48            for (int i = 0; i <= rear; i++) {
49                System.out.println((i + 1) + ". " + data[i]);
50            }
51        }
52    }
53
54    // cek kosong
55    boolean isEmpty_1014() {
56        return jumlah == 0;
57    }
58
59    // cek penuh
60    boolean isFull_1014() {
61        return jumlah == max;
62    }
63
64    // Reverse
65    void reverse_1014() {
66        if (isEmpty_1014()) {
67            System.out.println("Antrian kosong!");
68        } else {
69            Stack<String> stack_1014 = new Stack<>();
70
```

```

70
71     for (int i = 0; i <= rear; i++) {
72         stack_1014.push(data[i]);
73     }
74
75     for (int i = 0; i <= rear; i++) {
76         data[i] = stack_1014.pop();
77     }
78
79     System.out.println("Antrian berhasil dibalik");
80 }
81
82
83 public static void main(String[] args) {
84
85     Scanner input = new Scanner(System.in);
86     AntrianLoket_2511531014 q_1014 = new AntrianLoket_2511531014();
87
88     int pilih;
89     String isi;
90
91     do {
92         System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
93         System.out.println("1. Tambah Antrian");
94         System.out.println("2. Hapus Antrian");
95         System.out.println("3. Tampilkan Antrian");
96         System.out.println("4. Reverse Antrian");
97         System.out.println("5. Keluar");
98         System.out.print("Pilih menu : ");
99         pilih = input.nextInt();
100        input.nextLine();
101
102        switch (pilih) {
103            case 1:
104                System.out.print("Input data : ");
105                isi = input.nextLine();
106                q_1014.enqueue_1014(isi);
107                break;
108
109            case 2:
110                q_1014.dequeue_1014();
111                break;
112
113            case 3:
114                q_1014.display_1014();
115                break;
116
117            case 4:
118                q_1014.reverse_1014();
119                break;
120
121            case 5:
122                System.out.println("Program selesai");
123                break;
124
125            default:
126                System.out.println("Menu tidak tersedia");
127        }
128    } while (pilih != 5);
129 }
130
131 }
132
133
134

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class `AntrianLoket_2511531014` yang digunakan untuk mengimplementasikan struktur data Queue menggunakan array dalam kasus antrian loket. Pada bagian awal program dilakukan import `java.util.Scanner` untuk menerima input pengguna dan `java.util.Stack` yang digunakan pada proses pembalikan antrian.

1. Class ini memiliki atribut `max`, `data[]`, `front`, `rear`, dan `jumlah` yang digunakan untuk mengatur proses antrian. Variabel `max` menunjukkan kapasitas maksimum antrian, `data[]` digunakan sebagai tempat penyimpanan data, `front` menunjukkan posisi elemen terdepan, `rear` menunjukkan posisi elemen terakhir, sedangkan `jumlah` menyimpan banyaknya data yang ada di dalam antrian.

```

5
6 public class AntrianLoket_2511531014 {
7
8     int max = 10;
9     String data[] = new String[max];
10    int front = 0;
11    int rear = -1;
12    int jumlah = 0;
13

```

2. Method enqueue_1014(String isi) digunakan untuk menambahkan data ke dalam antrian. Sebelum penambahan, dilakukan pengecekan menggunakan isFull_1014() untuk memastikan antrian tidak penuh. Jika masih tersedia ruang, posisi rear bertambah satu, data dimasukkan ke array, dan jumlah data bertambah.

```

14 // Tambah Antrian
15 void enqueue_1014(String isi) {
16     if (isFull_1014()) {
17         System.out.println("Antrian penuh!");
18     } else {
19         rear++;
20         data[rear] = isi;
21         jumlah++;
22         System.out.println("Data berhasil ditambah");
23     }
24 }

```

3. Method dequeue_1014() digunakan untuk mengambil sekaligus menghapus data terdepan dari antrian. Jika antrian kosong, program menampilkan pesan kesalahan. Jika tidak kosong, data pada posisi depan ditampilkan, kemudian seluruh elemen digeser ke kiri agar posisi data tetap berurutan, lalu nilai rear dan jumlah dikurangi.

```

// Hapus Antrian
void dequeue_1014() {
    if (isEmpty_1014()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Data keluar: " + data[front]);

        for (int i = 0; i < rear; i++) {
            data[i] = data[i + 1];
        }

        rear--;
        jumlah--;
    }
}

```

4. Method display_1014() digunakan untuk menampilkan seluruh isi antrian. Jika antrian kosong, program menampilkan pesan bahwa antrian kosong. Jika tidak kosong, seluruh data dicetak secara berurutan dari depan hingga belakang menggunakan perulangan.

```

// Tampilkan Antrian
void display_1014() {
    if (isEmpty_1014()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Isi antrian:");
        for (int i = 0; i <= rear; i++) {
            System.out.println((i + 1) + ". " + data[i]);
        }
    }
}
}

```

5. Method isEmpty_1014() digunakan untuk mengecek apakah antrian kosong dengan memeriksa nilai jumlah, sedangkan method isFull_1014() digunakan untuk mengecek apakah antrian telah penuh sesuai kapasitas maksimum.

```

// cek kosong
boolean isEmpty_1014() {
    return jumlah == 0;
}

// cek penuh
boolean isFull_1014() {
    return jumlah == max;
}

```

6. Method reverse_1014() digunakan untuk membalik urutan data dalam antrian dengan bantuan struktur data Stack. Seluruh elemen antrian dimasukkan ke dalam stack menggunakan push(), kemudian diambil kembali menggunakan pop() dan disimpan ke array sehingga urutan data menjadi terbalik.

```

// Reverse
void reverse_1014() {
    if (isEmpty_1014()) {
        System.out.println("Antrian kosong!");
    } else {
        Stack<String> stack_1014 = new Stack<>();

        for (int i = 0; i <= rear; i++) {
            stack_1014.push(data[i]);
        }

        for (int i = 0; i <= rear; i++) {
            data[i] = stack_1014.pop();
        }

        System.out.println("Antrian berhasil dibalik");
    }
}

```

7. Pada method main, dibuat objek Scanner dan objek AntrianLoket_2511531014 sebagai antrian utama. Program menggunakan perulangan do-while untuk menampilkan menu secara berulang, dengan pilihan tambah antrian, hapus antrian, tampilkan antrian, reverse antrian, dan keluar dari program.

```

public static void main(String[] args) {

    Scanner input = new Scanner(System.in);
    AntrianLoket_2511531014 q_1014 = new AntrianLoket_2511531014();

    int pilih;
    String isi;

    do {
        System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Reverse Antrian");
        System.out.println("5. Keluar");
        System.out.print("Pilih menu : ");
        pilih = input.nextInt();
        input.nextLine();
    }

```

8. Struktur switch-case digunakan untuk menjalankan perintah sesuai pilihan pengguna. Setiap menu akan memanggil method yang sesuai sehingga proses pengelolaan antrian dapat dilakukan secara interaktif.

```

2 switch (pilih) {
3     case 1:
4         System.out.print("Input data : ");
5         isi = input.nextLine();
6         q_1014.enqueue_1014(isi);
7         break;
8
9     case 2:
10        q_1014.dequeue_1014();
11        break;
12
13    case 3:
14        q_1014.display_1014();
15        break;
16
17    case 4:
18        q_1014.reverse_1014();
19        break;
20
21    case 5:
22        System.out.println("Program selesai");
23        break;
24
25    default:
26        System.out.println("Menu tidak tersedia");
27
28    }
29    } while (pilih != 5);
30 }
31 }
32
33

```

9. Hasil Output dari Program

- Menambahkan Antrian dengan pilih menu 1

```
Console X
AntrianLoket_2511531014 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (Apr 29, 2026, 12:17:52 AM elapsed: 0:05:11)

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 1
Input data : haechan
Data berhasil ditambah

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 1
Input data : jeno
Data berhasil ditambah

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 1
Input data : jaemin
Data berhasil ditambah
```

- Untuk menampilkan antrian pilih menu 3 dan lakukan Reverse dengan pilih menu

```
Console X
AntrianLoket_2511531014 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (Apr 29, 2026, 12:17:52 AM elapsed: 0:05:11)

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 3
Isi antrian:
1. haechan
2. jeno
3. jaemin

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 4
Antrian berhasil dibalik

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu :
```

- Lanjutkan untuk menghapus antrian dengan pilih menu 2 dan Untuk keluar program dengan pilih menu 5

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse Antrian  
5. Keluar  
Pilih menu : 2  
Data keluar: jaemin
```

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse Antrian  
5. Keluar  
Pilih menu : 5  
Program selesai
```